

GROUP MEMBERS: Devin Pinyard (dmpinyar), Kevin Alvarenga (kaalvare)

Scenario:

One of my roommates, Jonathan, loves hot dogs. Unfortunately, far too often, he misplaces his hot dogs somewhere in the apartment. Now, Jonathan managed to lose his hot dogs again, and he needs to find them. Now, he only needs one pack of hot dogs, but somehow he's managed to lose his entire supply of three hot dogs! Further, he seems to have lost his tongs! He needs to find all of his hot dogs, and then one of his pairs of tongs afterward (after all, he can't pick up the hot dogs if he has tongs in his hands)! They seem to have been lost in some especially dark corner of the apartment this time. Jonathan cannot find them on his own using his antediluvian searching techniques, so we must help Jonathan find them by communicating a traditional searching algorithm to him by yelling at him from across the apartment. Hopefully, Jonny will be able to prepare his hot dogs with our help.

Testing:

All directional path costs are 1 unless otherwise specified. We chose "three different starting states" for our testing. We included a fourth test case that uses different edge weights as an additional test for cost comparison.

(BFS)

We report edge weight costs for BFS to show a comparison in solution cost with UCS, even though BFS doesn't use the edge weights. BFS uses a FIFO (queue) approach.

Test Case	# of Expanded States	Path Length	Total Path Cost (Edge Weights)
Start State: 32 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	220	14	13
Start State: 179 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	120	22	21
Start State: 143 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	120	14	13
Start State: 32 Hot Dogs: 62, 47, 82	220	14	76

Goals: 34, 48, 49 North Arc Cost: 10 South Arc Cost: 1 East Arc Cost: 10 West Arc Cost: 1			
---	--	--	--

(UCS)

UCS was implemented using a priority queue to optimize accumulated costs.

Test Case	# of Expanded States	Path Length	Total Path Cost
Start State: 32 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	220	14	13
Start State: 179 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	120	22	21
Start State: 143 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49	120	14	13
Start State: 32 Hot Dogs: 62, 47, 82 Goals: 34, 48, 49 North Arc Cost: 10 South Arc Cost: 1 East Arc Cost: 10 West Arc Cost: 1	222	14	76

Testing Reflection/Analysis:

Across the first three test cases, where all directional edge costs were equal to 1, BFS and UCS gave identical solution paths, lengths, and total path costs. This was expected since when all edge costs are uniform, minimizing the number of actions (BFS) would be the same as minimizing accumulated cost, $g(n)$, which is UCS. Therefore, both algorithms expanded the same number of states here, showing that UCS can be the same as BFS under the constraint of uniform edge costs. However, we did notice some differences in the weighted test case (case 4), where North

and East directions had an edge cost of 10, and South and West had an edge cost of 1. Both algorithms gave the same solution path and total cost, which means that for our current graph design, the lowest cost solution also happened to be the same as the shortest-step solution. This is mostly due to the walls and layout of our graph restricting certain areas and the placement of the hot dogs and tongs (goals). While both algorithms did provide the same solution, UCS expanded slightly more states, 222, while BFS expanded 220. This reflects the additional steps required by UCS to prioritize by cumulative cost and optimize path cost. On the otherhand, BFS just focused on the order of depth and doesn't take into account the edge weights for its search order.

From these results, we can infer that UCS will be advantageous in scenarios where action costs vary more and if there is a longer path that has a lower total cost than a shorter path. Our current graph design did not provide this tradeoff due to the restrictive nature of the walls and placement of items. In these cases, BFS could return a suboptimal solution since it doesn't take into account edge weights. If we were to implement A* instead, we would see a reduction in the number of states expanded while optimizing cost. This would be possible if we had an admissible heuristic, $h(n)$, that prioritized states that were closer to hot dogs and a goal.